

Starting 7:05

Agenda:-

1. Prefix Sum
2. Carry Forward
3. Subarrays

Q:-

Given an array representing the daily profit or loss from a particular stock over a period of days, write a function that calculates the total profit or loss over a given range of days. The function should efficiently handle multiple queries for different ranges without recalculating the sum for each query.

arr[] $\rightarrow$ {⁰-5, ¹10, ²20, ³40, ⁴35, ⁵-2, ⁶-10}

Q

Queries

<u>s</u>	<u>e</u>	
0	4	100
1	2	30
2	5	93
3	3	40

Idea 1:-

For each query, iterate on the array from start to end indices & accumulate the sum.
Print the sum.

Code

queries →

	0	1
0	0	4
1	1	2
2	2	5
3	3	4
4	2	2

```
void solve(int[] arr, int[][] queries) {  
    int s, e; int sum;  
  
    for (int q = 0; q < queries.length; q++) {  
        s = queries[q][0];  
        e = queries[q][1];  
  
        sum = 0;  
        for (int i = s; i <= e; i++) {  
            sum += arr[i];  
        }  
        print(sum);  
    }  
    return;  
}
```

$0 \rightarrow s \rightarrow e$
 $1 \rightarrow s \rightarrow e \dots N$
 $\vdots$
 $Q-1 \rightarrow s \rightarrow e \dots N$

T.C $\rightarrow O(N * Q)$

S.C $\rightarrow O(1)$

Scoreboard

Overs	1	2	3	4	5	6	7	8	9	10
Runs	2	8	14	29	31	49	65	79	88	97

Q:1

Only 7th Over

$$\begin{aligned} &= \text{Score}[7] - \text{Score}[7-1] \\ &= 65 - 49 = \underline{16}. \end{aligned}$$

Q:2

6th to 10th Over

$$\begin{aligned} &\text{Score}[10] - \text{Score}[6-1] \\ &= 97 - 31 = \underline{66}. \end{aligned}$$

Q:3

3rd to 6th

$$\begin{aligned} &\text{Score}[6] - \text{Score}[3-1] \\ &= 49 - 8 \\ &= \underline{41}. \end{aligned}$$

Observation

The scoreboard does not store the runs for each over, rather it stores the cumulative runs from 1st over till the i^{th} over & thus answering queries takes constant time.

How to create Prefix Sum array?

let $arr[] = \{2, 5, -1, 7, 1\}$

$pSum[i]$ $\rightarrow$ Total sum of array
from index 0 till index i

$pSum[0] \rightarrow arr[0];$

$pSum[1] \rightarrow arr[0] + arr[1];$

$pSum[2] \rightarrow arr[0] + arr[1] + arr[2];$

$\vdots$

$pSum[N-1] \rightarrow arr[0] + arr[1] \dots arr[N-1].$

$arr[] = \{2, 5, -1, 7, 1\}$

$pSum[] = \{2, 7, 6, 13, 14\}$

Code

```
int[] pSum (int[] arr) {  
    int pSum[] = new int[arr.size];  
    for (i = 0; i < arr.size(); i++) {  
        sum = 0;  
        for (j = 0; j ≤ i; j++) {  
            sum += arr[j];  
        }  
        pSum[i] = sum;  
    }  
    return pSum;  
}
```

$N(\frac{N+1}{2})$

T.C $\rightarrow O(N^2)$

i	j	# Iteration
0	[0,0]	1
1	[0,1]	2
2	[0,2]	3
⋮	⋮	⋮
N-1	[0,N-1]	N

Optimisation:-

$pSum[0] \rightarrow arr[0];$ $\nearrow pSum[0];$
 $pSum[1] \rightarrow arr[0] + arr[1];$
 $pSum[2] \rightarrow arr[0] + arr[1] + arr[2];$
 $\vdots$
 $\vdots$
 $pSum[N-1] \rightarrow arr[0] + arr[1] \dots arr[N-1].$
 $\downarrow$
 $pSum[N-2]$

arr: { 10, 32, 6, 12, 20, 13 }

Indices: 0 1 2 3 4 5

Arrows: $\nearrow \nearrow \nearrow \nearrow \nearrow$

pSum: { 10, 42, 48, 60, 80, 93 }.

$$\therefore \boxed{pSum[i] = arr[i] + pSum[i-1];}$$

$$\begin{array}{c} \nearrow \\ i-1=0 \\ \hline \end{array}$$

When $i == 0;$

$$\boxed{pSum[0] = arr[0];}$$

Optimised Code

```
int[] pSum (int[] arr) {  
    int pSum[] = new int[arr.size];  
    pSum[0] = arr[0];  
    for (i=1; i < N; i++) {  
        pSum[i] = arr[i] + pSum[i-1];  
    }  
    return pSum;  
}
```

T.C $\rightarrow O(N)$.

S.C $\rightarrow O(1)$.

Using Prefix Sum in original question:-

arr[] $\rightarrow$ {⁰-5, ¹10, ²20, ³40, ⁴35, ⁵-2, ⁶-10}
pSum[] $\rightarrow$ {-5, 5, 25, 65, 100, 98, 88}

Queries

<u>s</u>	<u>e</u>	
0	4	pSum[4] 100
1	2	pSum[2] - pSum[1-1] 30
2	5	pSum[5] - pSum[2-1] 93
3	3	pSum[3] - pSum[3-1] 40

$$[s, e] = \text{pSum}[e] - \text{pSum}[s-1]$$

else
 $s \neq 0$
 $\text{pSum}[e];$

Code

```
void solve (int[] arr, int[][] queries) {
```

```
    int pSum[] = pSum(arr);
```

```
    int s, e, sum;
```

```
    for (q=0; q < queries.size(); q++) {
```

```
        s = queries[q][0];
```

```
        e = queries[q][1];
```

```
        if (s == 0) {
            sum = pSum[e];
```

```
        } else {
            sum = pSum[e] - pSum[s-1];
```

```
        }
        print(sum);
    }
```

T.C $\rightarrow O(N+Q)$

S.C $\rightarrow O(N)$

↑
Because of
pSum array.

Q:-2. Given an array of size N & all queries, return the sum of all even indexed elements from start to end.

ex:-
 $\text{arr} = \{2, 3, 1, 6, 4, 5\}$
 $\text{pSumE} \rightarrow \{2, 2, 3, 3, 7, 7\}$

Queries

1	3	1	$\text{pSumE}[3] - \text{pSumE}[1-1] = 1$
2	5	5	$\text{pSumE}[5] - \text{pSumE}[2-1] = 5$
0	4	7	$\text{pSumE}[4] = 7$
3	3	0	$\text{pSumE}[3] - \text{pSumE}[3-1] = 0$

Idea:- Modify the prefix sum array to only consider the elements that are at an even index.

$\text{arr} = \{2, 3, 1, 6, 4, 5\}$

$\text{pSumE} \rightarrow \{2, 2, 3, 3, 7, 7\}$

$\text{pSumE}[i] = \text{pSumE}[i-1] + \text{arr}[i];$ if i is even
 $\text{pSumE}[i] = \text{pSumE}[i-1];$ if i is odd
 $\text{pSumE}[0] = \text{arr}[0];$

Q:-4 $\{2, 4, 3, 1, 5\}$

$\text{pSumE} = \{2, 2, 5, 5, 9\}$

Code

1. Generate a $psum$ array.

2. Answer every query using the formula.

$$[s, e] = psum_e[e] - psum_e[s-1]$$

$\checkmark s \neq 0$

else
 $psum_e[e]$.

Code $\rightarrow$ H.W.

Variation 1:-

Create $psum0$.

0 1 2 3 4 5
{ 2, 3, 1, 6, 4, 5 }

$psum0 = \{ 0, 3, 3, 9, 9, 14 \}$.

$$psum0[i] = psum0[i-1] + \begin{cases} 0 & i \text{ is even} \\ arr[i] & i \text{ is odd} \end{cases}$$

$$psum0[0] = 0;$$

8:25

To Remember:-

If you have a use case to iterate over a range of elements again & again, think about using the prefix sum approach.

Q:- 3 Given a string S of lowercase characters. Return the count of pairs (i, j) such that

1. $i < j$
2. $S[i] = 'a'$
3. $S[j] = 'g'$

ex:- $S = "a b e g a g"$

$(0, 3)$
 $(0, 5)$

$(4, 5)$

ans = 3;

Q:- 5.

$S = "a c g d g a g"$

$(0, 2)$
 $(0, 4)$

$(0, 6)$
 $(5, 6)$

ans = 4.

Count = 4
1
2
3
4
5

Q:- 6.

$S = "b c a g g a a g"$

$(2, 3)$
 $(2, 4)$
 $(2, 7)$

$(5, 7)$
 $(6, 7)$

ans = 5;

i
 j

Idea 1:- Run a loop to find an

'0'. When found, run another loop from $i+1$ to $N-1$ to count g's. When ever found, increment the count.

```
int solve (String s) {  
    int count = 0;  
    for (i = 0; i < s.length(); i++) {  
        if (s[i] == 'a') {  
            for (j = i+1; j < s.length(); j++) {  
                if (s[j] == 'g') {  
                    count++;  
                }  
            }  
        }  
    }  
    return count;  
}
```

3

"aaaaa"

TC $\rightarrow O(N^2)$

SC $\rightarrow O(1)$.

Idea 2:- Maintain a running count
/ carry forward the count of 'a's.
When 'g' is found, add that
count to the ans.

$S = "a^0 c^1 g^2 d^3 g^4 a^5 g^6"$ i

Count_a = 0, 1, 2

ans = 0 + 1 + 1 + 2

$S = "b^0 c^1 a^2 g^3 g^4 a^5 a^6 g^7"$ i

Count_a = 0, 1, 2, 3

ans = 0 + 1 + 1 + 3 = 5

Code

T.C $\rightarrow O(N)$

S.C $\rightarrow O(1)$

```
int solve (String s) {
```

```
    int ans = 0, count_a = 0;
```

```
    for (int i = 0; i < s.length(); i++) {
```

```
        if (s.charAt(i) == 'a') {
```

```
            count_a++;
```

```
        } else if (s.charAt(i) == 'g') {
```

```
            ans += count_a;
```

```
        }
```

```
    return ans;
```

```
}
```

Subarrays

A **Continuous** part of an array having **atleast 1** element.

ex:- arr :- $\{ \overset{0}{4}, \overset{1}{1}, \overset{2}{2}, \overset{3}{3}, \overset{4}{-1}, \overset{5}{6}, \overset{6}{9}, \overset{7}{8}, \overset{8}{12} \}$

- **2, 3, -1, 6** is a subarray of length 4.
- **9** is a subarray of single element.
- **4, 1, 2, 3, -1, 6, 9, 8, 12** is a subarray consisting of all the array elements.
- **4, 12** is **not** a subarray because loop does not count as subarray.
- **1, 2, 6** is **not** a subarray beacuse the array elements must be contiguous.
- **3, 2, 1, 4** is **not** a subarray because order of the elements in a subarray should be same as in the array.

Q:- 7

A[] = { 2, 4, 1, 6, -3, 7, 8, 4}
Which of the following is a valid subarray?

4 options
Active Duration(Most preferred: 30 seconds)

Appears for 45 Secs

- | | | |
|---|--------------|---|
| A | {1, 6, 8} | X |
| B | {1, 4} | X |
| C | {6, 1, 4, 2} | X |
| D | {7, 8, 4} | |

Representing a subarray:-

arr: { 4, 1, 2, 3, -1, 6, 9, 8, 12 }

Indices: 0 1 2 3 4 5 6 7 8

Two Ways:-

1. Starting & ending indices

$S = 3$
 $E = 6$

2. Starting Index & length. ; $S = 3$;
length = 4;

Total Subarrays

ex:-

arr: { 4, 2, 10, 3, 12, -2, 15 }

Indices: 0 1 2 3 4 5 6

	<u>Starting Index</u>	<u>E.I.</u>	<u>Count</u>
<u>N = 7</u>	0	[0, 6]	7
<u>N = 6</u>	1	[1, 6]	6
	2	[2, 6]	5
	3	[3, 6]	4
	4	[4, 6]	3
	5	[5, 6]	2
	6	[6, 6]	1

∴ For any N. $\frac{7 \times (7+1)}{2} = 28$

Total Subarrays = $\frac{N(N+1)}{2}$

Q:- Print all subarrays of an array:-

ex:- arr = ⁰1, ¹2, ²3

<u>o/p:-</u>		<u>o/p</u>
	0, 0	1
	0, 1	1 2
	0, 2	1 2 3
	1, 1	2
	1, 2	2 3
	2, 2	3

Idea:- Consider a subarray by fixing the starting & ending indices of the array. Iterate on that subarray to print that element.

```
for (s = 0; s < N; s++) {  
    for (e = s; e < N; e++) {  
        for (i = s; i <= e; i++) {  
            print(arr[i]);  
        }  
        println();  
    }  
}
```

3
3
3

T.C $\rightarrow O(N^3)$